

Knowledge Graph Extraction from Plant Pest Documents Using BioBERT-based NER and Marker-based Relation Classification^{*}

Sree Krishna S at PestCLEF 2026

Sree Krishna S¹, Varghese K James¹, Sujith M¹ and Prabavathy Balasundaram¹

¹Sri Sivasubramaniya Nadar College of Engineering, Chennai, India

Abstract

We describe the Sree Krishna S system submitted to the PestCLEF 2026 shared task, which frames Knowledge Graph (KG) extraction from plant pest web documents as a two-stage pipeline: Named Entity Recognition (NER) followed by relation classification. Our approach fine-tunes BioBERT for token-level entity tagging across seven domain-specific entity types, exploiting its pre-training on biomedical literature to handle the dense scientific nomenclature of the EPOP corpus. A marker-based BioBERT relation classifier then inserts special boundary tokens around candidate entity spans, allowing the encoder to produce context-aware representations anchored to each entity's position in the text rather than relying on generic sentence-level pooling. Schema-based filtering further constrains candidate pairs to biologically valid entity-type combinations, eliminating implausible relations without any model computation. We describe the iterative development of three system versions, each correcting a fundamental data-handling error from the previous one, culminating in a pipeline that correctly trains on gold annotations from the EPOP corpus. Our final system achieves a macro-averaged F1 of 0.1403 on the development set and 0.1456 on the private test set, placing 10th on the final competition leaderboard. The NER component converges to an F1 of 0.622, demonstrating the viability of the BioBERT fine-tuning approach for this domain.

Keywords

knowledge graph extraction, named entity recognition, relation extraction, BioBERT, plant pest monitoring, PestCLEF 2026

1. Introduction

Automated Extraction of structured ecological knowledge from natural language documents is a challenge for plant health surveillance. Monitoring spread of crop pests, disease outbreaks and vector distributions across different parts of the world requires large volumes of heterogeneous web documents to be processed. It is not feasible for a human to process such large volumes of text. PestCLEF 2026 [1] is a shared task in the LifeCLEF evaluation lab at CLEF 2026 that addresses this challenge directly.

Our system, submitted under the team identifier Sree Krishna S, adopts a two-stage pipeline architecture. In the first stage, a BioBERT token classifier identifies entity spans and their types. In the second stage, a marker-based BioBERT relation classifier predicts the predicate for each candidate entity pair produced by stage 1, based on the entity-type combinations defined in the task schema. The development of this system involved three distinct iterations, each correcting a flaw in how training data was loaded and used. We document all three versions, as the mistakes are useful for practitioners building similar pipelines

CLEF 2026 Working Notes, 21 – 24 September 2026, Jena, Germany

^{*} Submitted as a working note to the PestCLEF 2026 shared task at LifeCLEF, CLEF 2026. Code and notebooks are publicly available at <https://github.com/SSK166/PestClefFSK2026>.

✉ sreekrishnassk166@gmail.com (S. K. S); varghesejk18@gmail.com (V. K. James); Sujithmaris@gmail.com (S. M); prabavathyb@ssn.edu.in (P. Balasundaram)

🌐 <https://github.com/SSK166> (S. K. S); <https://github.com/vicfic18> (V. K. James); <https://github.com/sujith0613> (S. M); <https://www.ssn.edu.in/computer-science-and-engineering-department/faculty/dr-b-prabavathy-associate-professor/> (P. Balasundaram)

🆔 0009-0000-2362-165X (S. K. S); 0009-0002-9977-768X (V. K. James); 0009-0001-0671-6195 (S. M); 0000-0003-3903-0410 (P. Balasundaram)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

Our final submitted system achieves a macro-averaged F1 of 0.1456 on the private test set (approximately 82% of test documents), placing 10th on the final PestCLEF 2026 competition leaderboard. All code, training notebooks, and the final submission script are publicly available at <https://github.com/SSK166/PestClefSK2026>.

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 describes the task and dataset. Section 4 describes the evolution of our system across three versions. Section 5 presents the final system architecture. Section 6 describes experimental settings. Section 7 reports results and analysis. Section 8 describes how to reproduce our results. Section 9 concludes with directions for future work.

2. Related Work

2.1. Biomedical Named Entity Recognition

Pre-trained language models have substantially advanced the state of the art in biomedical NER. Lee et al. introduced BioBERT [2], a domain-specific BERT model pre-trained on PubMed abstracts and PubMed Central full texts. BioBERT demonstrated consistent improvements over general-domain BERT on three biomedical benchmarks: NER, relation extraction, and question answering, with the NER improvement of 0.62% F1 attributed to the model’s exposure to biomedical terminology and Latin species names during pre-training. We used BioBERT for the NER stage of the pipeline as the EPOP corpus contains dense scientific nomenclature for plant pathogens and host species.

2.2. Relation Extraction with Entity Markers

A key design choice in our system is the use of entity boundary markers for relation classification. This technique was introduced by Baldini Soares et al. [3], who proposed inserting special tokens at the boundaries of each entity span so that the BERT encoder can attend directly to the entity positions rather than pooling a generic sentence-level representation. Their experiments on FewRel, SemEval 2010 Task 8, and TACRED demonstrated that the entity-start representation (the hidden state at the opening marker) significantly outperforms CLS-based pooling for relation classification. While the original method uses only the subject-start hidden state, our implementation concatenates the hidden states at both the subject-start and object-start markers to represent both entities and we further append three numbers to this combined representation: the normalised token distance between the two markers, and the normalised absolute positions of each marker in the sequence, providing the classifier with positional context about how far apart the two entities appear in the document.

Pipeline-based relation extraction — where NER and relation classification are performed sequentially as separate models — is a paradigm reviewed in depth by Zhao et al. [4]. While joint models can reduce error propagation from the NER stage, pipeline approaches remain competitive when annotated data is limited, as independent training allows each component to be tuned separately. We opted for the pipeline approach as the EPOP training set is small (110 documents).

2.3. Agricultural Knowledge Graph Construction

Knowledge graph construction from agricultural texts has received growing attention [5, 6]. Most relevant to our work, Bossy and Nédellec [7] found that even the state of the art LLMs struggle with the relational structure of the EPOP dataset — a challenge addressed by our pipeline.

3. Task and Dataset

3.1. Task Definition

PestCLEF 2026 requires participants to extract a set of KG triples representing ecological interactions involving plant pests, for each input document. Each triple consists of a subject entity, a predicate, and

an object entity. The evaluation metric is macro-averaged F1 over all documents in the test set, where per-document F1 is computed as the harmonic mean of precision and recall over the set of predicted triples matched against the gold KG.

3.2. The EPOP Corpus

The EPOP corpus [1] comprises 247 web documents focused on 20 monitored pest species, split into training (110 documents), development (55 documents), and test (82 documents) sets. Documents vary considerably in length and genre. Each document is annotated with named entities, relations between entities, and a document-level KG.

3.3. Entity and Relation Schema

The task defines seven entity types: *Pest*, *Plant*, *Disease*, *Vector*, *Dissemination_pathway*, *Location*, and *Date*. Seven relation predicates are defined (excluding *NO_RELATION*): *Affects*, *Causes*, *Dispersed_by*, *Found_on*, *Located_in*, *Occurs_on*, and *Transmits*. Not all subject–object type pairs are valid for each predicate; the schema defines 17 valid (subject type, object type) combinations. For example, only a *Vector* can be the subject of a *Transmits* relation, and only a *Disease* or *Pest* can be its object.

4. System Development

The three versions of the pipeline are described below.

4.1. Version 1: Silver-Label-Only Pipeline

The first version loaded only the raw `.txt` document files and ignored the gold JSON annotations entirely. Because no gold entity annotations were available to the model, the NER training relied exclusively on silver labels generated by rule-based term dictionary matching over the document text. While this approach is a reasonable strategy for the test set (where no gold annotations exist), applying it to the training and development splits discards the expert-annotated entity spans provided by the EPOP corpus.

A second critical flaw in this version was a severely restricted relation pair generation function. The `generate_pairs` function hard-coded a constraint that only pairs of the form (*any entity type*, *Location*) were considered as candidate relations. This eliminated over 80% of valid relation types from consideration entirely, since most EPOP relations involve Pest–Plant, Vector–Disease, and Pest–Dissemination_pathway combinations that do not include Location as the object.

The combined effect was a model trained on noisy, incomplete annotations that could only predict a narrow subset of the required predicates.

4.2. Version 2: Attempted Gold Integration with Parsing Errors

The second version attempted to incorporate the gold JSON annotations by adding a `load_gold_json()` function. However, this function used incorrect field names to parse the EPOP JSON format. Specifically, it expected fields named `id`, `text`, `entities`, and `relations` at the top level, whereas the actual EPOP schema uses `doc_id`, `text_bound_annotations`, and a nested structure for entities and relations with character-offset arrays.

Additionally, the relation argument role mapping was not implemented. The EPOP relation format encodes subject and object slots through role names (Object, Organism, Nuisance for subjects; Location, Habitat, Date, Dissemination_pathway for objects) rather than explicit subject/object fields. Without this mapping, all relations parsed to empty or incorrect entity pairs.

The result was that the gold annotations were nominally loaded but effectively empty: the NER model still trained on silver labels and the relation model trained on no positive examples extracted from the gold KG.

Table 1

Comparison of the three system development versions.

Component	Version 1	Version 2	Version 3 (final)
Data loader	txt-only	JSON (wrong fields)	JSON (correct)
NER labels	silver (all)	silver (all)	gold train/dev
RE training data	silver pairs	empty	gold relations
Valid pairs	Location-only	Location-only	full schema
Silver labeling	always	always	test-only fallback

4.3. Version 3: Correct Gold Integration (Final Submission)

The final version rewrote the data loading pipeline from scratch with a unified `load_json_split()` function that correctly handles the EPOP JSON schema. Key fixes included: reading the `doc_id` field; parsing `text_bound_annotatations.entities` with their `offsets` arrays for precise character-level span extraction; resolving relation argument roles to subject and object slots using the correct role-name mapping; and parsing the `knowledge_graph` edges, where subject and object are stored as arrays of accepted surface forms with the first form used as the canonical string.

Silver labeling was demoted to a test-set-only fallback, applied only when a document has no gold `text_bound_annotatations`. The relation pair generation was replaced by the full `VALID_PAIRS` schema check, enabling the model to learn all seven predicate types. This version is described in full in the following section.

Table 1 summarises the key differences across the three development versions.

5. Final System Architecture

Our final system consists of three components: a BioBERT NER model, a marker-based BioBERT relation classifier, and a KG assembly module.

5.1. Stage 1: Named Entity Recognition

Model. We fine-tune `dmis-lab/biobert-base-cased-v1.2` for token classification using the Hugging Face `transformers` library [2]. The model produces a distribution over 15 BIO entity labels (B- and I- prefixes for each of the seven entity types, plus O) for every input token. The token-level classification head is a linear layer initialised randomly and trained from scratch on top of the frozen-then-unfrozen BioBERT encoder.

Input preparation. Raw document text is pre-processed by collapsing repeated whitespace and removing very short lines. Gold entity annotations, provided as character offsets, are converted to word-level BIO labels by matching each entity span to whitespace-tokenised words. Sub-word tokenisation by BioBERT’s WordPiece tokeniser is handled by aligning labels to the first sub-word of each word and masking continuation tokens with -100 during loss computation.

Long document handling. Because EPOP documents frequently exceed BioBERT’s 512-token context limit, we apply sliding-window inference with a window size of 512 tokens and a stride of 462 tokens. Window predictions are stitched back to the full token sequence by position, with later windows overwriting earlier ones in the overlap region.

Class imbalance. Entity tokens constitute approximately 12.4% of training tokens; the remaining 87.6% are labelled O. To counteract this imbalance we use weighted cross-entropy loss, computing inverse-frequency weights per class and capping them at $5\times$ to avoid extreme over-correction. Training

uses the `WeightedTrainer` class, a thin subclass of the Hugging Face `Trainer` that overrides the loss computation.

Inference and filtering. At inference time, predicted entity spans with a confidence score below 0.45 are discarded, as are spans shorter than three characters and pure-digit tokens. A curated blocklist of generic geographic terms (*global, world, country, international, area, province, recently*) is used to suppress common false positives from the Location and Date entity classes. Duplicate spans of the same type are deduplicated, and the entity list is capped at 50 entities per document to limit the quadratic growth of candidate pairs in the subsequent stage.

5.2. Stage 2: Relation Classification

Model. The relation classifier uses a custom `RelationClassifier` module built on top of `dmis-lab/biobert-base-cased-v1.2`. For each candidate entity pair (subject s , object o), the model receives a modified version of the document text with four special boundary tokens inserted:

```
[SUBJ_START] <subject span> [SUBJ_END] ... [OBJ_START] <object span>
[OBJ_END]
```

This marker-based encoding, introduced by Baldini Soares et al. [3], allows the BioBERT encoder to attend directly to the entity boundaries. The hidden states at the `[SUBJ_START]` and `[OBJ_START]` positions are extracted from the final encoder layer and concatenated to form a 1536-dimensional representation. Three scalar distance features are appended: the normalised absolute token distance between the two marker positions, and the normalised absolute positions of each marker in the sequence. The resulting 1539-dimensional vector is passed through the classifier head:

`Dropout(0.15) → Linear(1539 → 768) → GELU → LayerNorm(768) → Dropout(0.15) → Linear(768 → 8)`

The output is an 8-dimensional logit vector (one per predicate including `NO_RELATION`). Softmax probabilities are computed at inference; the predicted label is the argmax and its probability is the confidence score.

Schema-based candidate filtering. Before encoding any pair, the system checks whether the (subject type, object type) combination appears in the `VALID_PAIRS` schema. Pairs not in the schema are skipped entirely. This constraint eliminates biologically implausible candidates without consuming model compute, and substantially reduces the false positive rate.

Training data construction. Positive examples are extracted directly from the gold relation annotations in the training and development splits. Hard negative examples are constructed by enumerating schema-valid entity pair permutations that do not appear as positives within the same document and whose entity mentions are within 300 characters of each other. Hard negatives are sampled at a ratio of 3:1 negative-to-positive per document. The final training set contains 7,758 examples: 2,874 positive and 4,884 negative.

Class weighting. Predicate frequency varies substantially across the training set. We compute log-normalised inverse-frequency weights (`torch.log1p`) and clip them to the range `[0.5, 10.0]`. The resulting weights give the rare predicates (*Causes, Transmits, Affects*) weights near 8.79 and the majority class (`NO_RELATION`) a weight near 0.95.

5.3. Stage 3: KG Assembly

After relation prediction, candidate edges with a confidence score below 0.45 are discarded. Remaining edges are passed to `build_kg()`, which applies a secondary threshold of 0.35, resolves entity surface forms to canonical IDs, normalises edge keys (lowercased subject and object, underscore-stripped predicate), and deduplicates using a seen-set. The final KG for each document is serialised as a JSON array of {subject, predicate, object} triples and written to the submission CSV.

Test-set fallback. Because the test set has no gold entity annotations, entities are generated by a rule-based silver labeller: curated term dictionaries for each entity type are matched against the document text using case-insensitive regular expressions, with longer terms taking priority to reduce partial-match noise. This provides entity candidates for the relation model even where the NER model may produce sparse output.

6. Experimental Setup

6.1. Training Configuration

NER model. We train for 8 epochs with batch size 16, learning rate 3×10^{-5} , weight decay 0.01, and a linear warmup over the first 10% of total training steps. Mixed-precision training (FP16) is used on GPU. The best checkpoint is selected by development-set F1 at the end of each epoch.

Relation model. We train for 5 epochs with batch size 16, learning rate 2×10^{-5} , weight decay 0.01, and gradient clipping at 1.0. A linear learning rate schedule with 10% warmup is applied. The best checkpoint across epochs is saved based on macro-averaged F1 over positive predicates on a held-out 15% of the training examples (rounded to 1,164 validation examples, with 6,594 used for training).

6.2. Hardware

All experiments were conducted on Google Colab with an NVIDIA T4 GPU (16 GB VRAM). BioBERT was loaded from the Hugging Face model hub (`dmis-lab/biobert-base-cased-v1.2`). Approximate wall-clock times were 55–70 minutes for NER training (8 epochs) and 30–40 minutes for relation model training (5 epochs).

7. Results and Analysis

7.1. NER Performance

Table 2 shows the NER model’s performance across epochs. The model improves steadily to an F1 of 0.622 at the final epoch, with precision 0.544 and recall 0.726 at the best checkpoint. The entity token density of 12.4% (6,568 entity tokens out of 52,947 total) indicates that the EPOP documents contain a reasonably high concentration of entities, validating the use of weighted loss.

7.2. Relation Model Performance

The relation classifier achieves a best macro-averaged F1 of 0.4145 on the validation split (epoch 4 of 5). Training loss decreases consistently from 1.0525 to 0.3767, indicating that the model is learning well. The class weights assigned to rare predicates (up to $8.79\times$) successfully prevent the model from predicting NO_RELATION for all inputs during training.

Table 2

NER development-set performance per epoch.

Epoch	Precision	Recall	F1
1	0.000	0.000	0.000
2	0.438	0.092	0.153
3	0.438	0.460	0.449
4	0.473	0.635	0.542
5	0.470	0.671	0.553
6	0.513	0.697	0.591
7	0.544	0.716	0.619
8	0.544	0.726	0.622

Table 3

Summary of development-set KG evaluation.

Metric	Value
Total documents	55
Documents with F1 = 0.0	46
Documents with partial F1 (>0 , <1)	5
Documents with F1 = 1.0	4
Total predicted edges	125
Average edges per document	2.27
Macro-averaged F1	0.1403

7.3. Development Set KG Evaluation

Table 3 summarises the pipeline’s performance on the 55 development documents. The system produces 125 total edges across 55 documents (average 2.27 edges per document), achieving a macro-averaged F1 of **0.1403**.

7.4. Test Set Result

On the private test set (approximately 82% of test documents), the system achieves a macro-averaged F1 of **0.1456**, placing **10th** on the final PestCLEF 2026 competition leaderboard. The close agreement between development F1 (0.1403) and test F1 (0.1456) indicates that the pipeline generalises consistently across splits and does not overfit to the development set.

7.5. Analysis

The schema-based filtering removes invalid entity-type pairs before any model computation, reducing false positives. The marker-based encoding directs the BioBERT encoder to focus on entity boundaries rather than the whole sentence, which matters in EPOP documents where multiple entities of the same type appear close together. The weighted loss in both models prevents them from always predicting the most common class, as seen in the NER recall of 0.726 and the relation model F1 of 0.4145

The gap between per-component performance and end-to-end KG F1 is characteristic of pipeline systems where errors propagate between stages: entity spans not recovered by NER cannot contribute to any relations, and the final KG F1 is bounded above by the NER recall. The consistent performance across development and test sets confirms that the iterative correction of data loading errors across three system versions was the critical factor in arriving at a functional pipeline.

7.6. Discussion

The marker-based BioBERT approach offers several advantages for this domain. First, it requires no additional feature engineering beyond inserting four special tokens per candidate pair. Second, the valid entity-type pairs are defined in a simple lookup table that domain experts can easily inspect and modify. Third, independent training of NER and RE components allows each to be improved separately in future work without retraining the entire system.

We also found that many short documents (fewer than 200 words) produced no entities above the confidence threshold, resulting in an empty KG. Adjusting the confidence threshold for short documents is a key direction for future work.

8. Reproducibility

To facilitate reproduction of our results, we provide all code and notebooks in a public GitHub repository:

<https://github.com/SSK166/PestClefSK2026>

The repository contains three notebooks corresponding to the system versions described in Section 4:

- `PestClefSK2026-work-f1.ipynb` — The first version of the notebook; uses txt-only loading and silver labels throughout. Included for reference.
- `PestClefSK2026_latest.ipynb` — The second version of the notebook; contains the corrected `load_json_split()` loader and gold annotation integration.
- `PestClefSubmissionSKVSH.ipynb` — This is the script that was used to produce the final submission.

Data setup The EPOP corpus is distributed through the PestCLEF 2026 Kaggle competition page. After accepting the licence agreement, download `PestCLEF-2026_train.json`, `PestCLEF-2026_dev.json`, and `PestCLEF-2026_test.json`, together with the corresponding `.txt` document files. Place them under a Google Drive folder structured as follows:

```
EPOP_documents /
  goldKG /
    train.json
    dev.json
    test.json
  train/    <-- .txt files for training documents
  dev/      <-- .txt files for development documents
  test/     <-- .txt files for test documents
```

Update `BASE_DIR` in the notebook to point to this folder.

Environment. The submission was produced on Google Colab with a T4 GPU runtime. Required packages (installed via the first notebook cell):

```
pip install seqeval transformers datasets accelerate scikit-learn
      tqdm rapidfuzz
```

All random seeds are fixed to 42. Training involves stochastic elements (mini-batch shuffling, dropout); minor numerical variation across runs is expected, but results within ± 0.005 F1 of the reported figures should be reproducible on equivalent hardware.

9. Conclusion

We presented the Sree Krishna S system for PestCLEF 2026, a two-stage pipeline combining BioBERT-based NER with marker-based relation classification for Knowledge Graph extraction from plant pest documents. We documented the iterative development of three system versions, showing how incorrect corpus parsing in early versions prevented any meaningful learning from the gold EPOP annotations, and how correcting the data loading pipeline in the final version produced a consistent and generalisable system.

The final system achieves a NER token-level F1 of 0.622, a relation classification validation F1 of 0.4145, a development-set KG macro F1 of 0.1403, and a private test-set macro F1 of 0.1456, placing 10th on the final competition leaderboard. The close match between development and test scores demonstrates that the pipeline generalises well across document splits.

Future work will focus on three directions: (i) lowering and cross-validating the relation confidence threshold, particularly for short documents that currently produce empty KGs; (ii) replacing sentence-level relation training with full-document or cross-sentence encoding to better capture the long-range ecological relations characteristic of EPOP documents; and (iii) incorporating entity normalisation to reduce mismatches between different surface forms of the same entity during KG evaluation.

Acknowledgments

The authors thank the PestCLEF 2026 organisers at INRAE Paris-Saclay for providing the EPOP corpus and the evaluation infrastructure.

Declaration on Generative AI

During the preparation of this work, the author(s) used Claude (Anthropic): for grammar checking, code documentation, and preparation of this working note. After using this tool, the author(s) reviewed and edited the content as needed and take full responsibility for the publication's content.

References

- [1] R. Bossy and C. Nédellec. PestCLEF 2026: Shared Task on Knowledge Graph Extraction on Plant Pests from Web Documents. In *Proceedings of the CLEF 2026 Working Notes*, CEUR Workshop Proceedings, Jena, Germany, 2026.
- [2] Jinhyuk Lee, Wonjin Yoon, Sungdong Kim, Donghyeon Kim, Sunkyu Kim, Chan Ho So, and Jaewoo Kang. BioBERT: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 36(4):1234–1240, 2020.
- [3] Livio Baldini Soares, Nicholas FitzGerald, Jeffrey Ling, and Tom Kwiatkowski. Matching the blanks: Distributional similarity for relation learning. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2895–2905, Florence, Italy, July 2019. ACL. doi:10.18653/v1/P19-1279.
- [4] Weinan Zhao, Xin Ye, Meishan Yang, Zhenjun Lei, Zhengdao Zhang, and Rong Zhao. A comprehensive survey on deep learning for relation extraction: Recent advances and new frontiers. *ACM Computing Surveys*, 2024. doi:10.1145/3674501.
- [5] Kun Wang, Yifei Miao, Xin Wang, Yu Li, Fang Li, and Hao Song. Research on the construction of a knowledge graph for tomato leaf pests and diseases based on the named entity recognition model. *Frontiers in Plant Science*, 15:1482275, 2024. doi:10.3389/fpls.2024.1482275.
- [6] Hao Wu, Ning Xie, Xin Wang, Jian Fan, Yan Li, and Zhongwei Meng. Crop GraphRAG: pest and disease knowledge base Q&A system for sustainable crop protection. *Frontiers in Plant Science*, 16:1696872, 2026. doi:10.3389/fpls.2025.1696872.

- [7] Claire Nédellec et al. Consistency–accuracy correlation in hard-prompted LLMs for entity and relation extraction: empirical findings from plant-health data. *PubMed Central*, 2025.